



INTEGRAL UNIVERSITY LUCKNOW

Dasauli, Bas-ha Kursi Road, Lucknow – 226026

Department of Computer Application

LAB MANUAL

Session : 2021-2022

Subject Name : JAVA PROGRAMMING LAB

Subject Code : CA216

Branch : BCA

Year : 2nd

Semester : 4th

Revision History

Revision History					
Revision	Author(s)	Reviewer(s)	Date of Release	Owner	Summary of Changes
Revision -1	Mohammad Isha Mansoori	Internal Reviewers	07/03/2022	HOD - CA	Initial Release
Revision -2	Mr. Wasim Khan	Departmental QAC	12/03/2022	HOD - CA	Major revisions made. Incorporated review comments from departmental QAC into Revision-2

Vision

- To lead the teeming millions of the world through the wilderness of ignorance and illiteracy, as "Kindly Light" (Exodus 13:21) with the resounding divine proclamation. "Read: Thy Lord is the most bounteous (Quran 30:96:3)." and to educate them in the most constructive and innovative ways.
- To integrate the ebullience, intellect and dynamism of youth with decency, decorum, discipline and dedication through value-based quality education.

Mission

- To make every student a role model of intellectuals and torch bearer for others all over the world through his / her inspiring existence.
- To inculcate a spirit of confidence, self-respect and firm commitment in students along with farsighted wisdom and understanding.
- To make India a self-reliant and dominant G-1 country recognized for quality education, higher economic growth and valuable moral practices.

Programme Educational Objectives (PEO)

- To acquaint students about principles of system analysis, design, development and project management.
- To impart knowledge about various sub domains related to the field of computer science and applications.
- To apply IT practices to model and analyze the real-life problems and interpret the results.
- To build and lead cross-functional teams, upholding the professional responsibilities & ethical values.

Programme Outcomes (PO)

- Ability to demonstrate knowledge of Computer science and its applications in order to enhance basic understanding of various software technologies.
- Ability to analyze and identify various business and technical problems to further solve problems with effective communication.
- Ability to adapt analytical, logical and managerial skills with the technical aspects in order to design and deploy reliable software programs and application for real world problems.
- Ability to investigate complex problems and provide computer-based solutions.
- Ability to understand and deliver ethical, social and cultural responsibilities in professional environment as an individual and team.
- Ability to adapt new technologies for upgrading their skills and contributing to a lifelong learning.
- Ability to create and manage multidisciplinary projects and successfully apply software and project management principles.
- Ability to become employable in a variety of IT companies and government sector and also seek entrepreneurship opportunities for the betterment of an individual and society at large.

Course Objectives: (CA216 Java Programming Lab)

- 1- To introduce the object-oriented programming concepts, polymorphism and Inheritance using JAVA and apply them in solving problems.
- 2- Gain knowledge about the principles of interface and also introduce the use of super, final and this key word in JAVA.
- 3- To introduce the concepts of Package and Multithreading in JAVA programming.
- 4- To learn the Concept of Exception handling and Applets.
- 5- To introduce the concept of Pannel and Layout, incorporating graphics working with color and Fonts

Course outcome

After undergoing this laboratory module, the participant should be able to:

- CO1. Able to implement Object oriented, Polymorphism and Inheritance concepts in JAVA.
- CO2. Able to develop JAVA applications using the concepts of interface, super and final.
- CO3. Able to develop JAVA applications using multithreading and implementation of packages and Interfaces in JAVA.
- CO4. Able to develop java applications using the concept of Exception handling.
- CO5. Able to develop web-based applications using Applets.

Guidelines for the Students

1. The students should implement the given Program individually.
2. While conducting the experiments students should see that their programs would Meet the following criteria:
 - I. Programs should be interactive with appropriate prompt messages, error messages if any, and descriptive messages for outputs.
 - II. Programs should perform input validation (Data type, range error, etc.) and give appropriate error messages and suggest corrective actions.
 - III. Comments should be used to give the statement of the problem and every function should indicate the purpose of the function, inputs and outputs
 - IV. Statements within the program should be properly indented.
 - V. Use meaningful names for variables and functions.
 - VI. Make use of Constants and type definitions wherever needed.
3. Once the experiment(s) get executed, they should show the program and results to the instructors and copy the same in their Lab Report.
4. Students should be regular and come prepared for the lab practice.
5. In case a student misses a class, it is his/her responsibility to complete that missed experiment(s).
6. Students should bring the lab Report, class notes can be kept ready for reference if required.

List of Lab Experiments

Week No	Experiment No.	Name of the Experiments	Page No.
Week-1			
1	1	Introduction and history of Java	8
Week-2			
2	2	Program illustrating Classes and Objects.	17
	3	Constructor and Method Overloading	
Week-3			
3	4	Concept of inheritance and abstraction.	28
	5	Programs of interface and overriding.	
Week-4			
4	6	Program illustrating use of Final and Super keyword..	37
	7	Program that illustrates the following a) Creation of simple package. b) Accessing a package	
Week-5			
5	8	Program for creating multiple threads a) Using Thread class. b) Using Runnable Interface	45
Week-6			
6	9	Program that illustrates the following a) Handling predefined exceptions. b) Handling user defined exceptions.	51
Week-7			
7	10	Working with Applet, Panel and Layout.	60
Week-8			
8	11	Incorporating Graphics.	68

Week-1

OBJECTIVE: Introduction and history of java, concept and program of class and object.

Java:

Java is a programming language and a platform. Java is a high level, robust, object-oriented and secure programming language.

Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995. James Gosling is known as the father of Java. Before Java, its name was Oak. Since Oak was already a registered company, so James Gosling and his team changed the Oak name to Java.

JAVA introduction: -

Author	:	<i>James Gosling</i>
Vendor	:	<i>Sun Micro System (which has since</i>
Project name	:	<i>Green Project</i>
Type	:	<i>open source & free software</i>
Initial Name	:	<i>OAK language</i>
Present Name	:	<i>java</i>
Extensions	:	<i>.java & .class & .jar</i>
Initial version	:	<i>jdk 1.0 (java development kit)</i>
Present version	:	<i>java 8 2014</i>
Operating System	:	<i>multi-Operating System</i>
Implementation Lang	:	<i>c, cpp.....</i>
Symbol	:	<i>coffee cup with saucer</i>
Objective	:	<i>To develop web applications</i>
SUN	:	Stanford Universally Network
Slogan/Motto	:	<i>WORA (write once run anywhere)</i>

Importance of core java:-

According to the SUN 3 billion devices run on the java language only.

- 1) Java is used to develop Desktop Applications such as Media Player, Antivirus etc.
- 2) Java is Used to Develop Web Applications such as sravyajobs.com, irctc.co.in etc.
- 3) Java is Used to Develop Enterprise Application such as Banking applications.
- 4) Java is Used to Develop Mobile Applications.
- 5) Java is Used to Develop Embedded System.
- 6) Java is Used to Develop Smartcards.
- 7) Java is Used to Develop Robotics.
- 8) Java is used to Develop Games *etc.*

Features of Java

1- Simple: -

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

2- Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporates both data and behavior. Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

Object

Class

Inheritance

Polymorphism

Abstraction

Encapsulation

Platform Independent

Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.

There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. It has two components:

1. Runtime Environment
2. API (Application Programming Interface)

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).

Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- No explicit pointer
- Java Programs run inside a virtual machine sandbox

Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

The Software requirement for Java program

For executing any Java program, the following software or application must be properly installed.

- Install the JDK if you don't have installed it, download the JDK and install it.
- Set path of the jdk/bin directory.
- Create the Java program.
- Compile and run the Java program.

How to set path in Java

The path is required to be set for using tools such as javac, java, etc.

If you are saving the Java source file inside the JDK/bin directory, the path is not required to be set because all the tools will be available in the current directory.

However, if you have your Java file outside the JDK/bin folder, it is necessary to set the path of JDK.

There are two ways to set the path in Java:

- Temporary
- Permanent

1) Setting Temporary Path of JDK in Windows

To set the temporary path of JDK, you need to follow the following steps:

- Open the command prompt
- Copy the path of the JDK/bin directory
- Write in command prompt: set path=copied_path

For Example

```
set path=C:\Program Files\Java\jdk1.6.0_23\bin
```

Let's create the hello java program:

```
class First
{
    public static void main(String x[])
    {
        System.out.println("Hello this is my first java programme");
    }
}
```

Save the above file as First.java.

To compile:

```
javac First.java
```

To execute:

java First

Output:

Hello this is my first java programme

Parameters used in First Java Program:

Let's see what is the meaning of class, public, static, void, main, String[], System.out.println().

- **class** keyword is used to declare a class in Java.
- **public** keyword is an access modifier that represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main() method is executed by the JVM, so it doesn't require creating an object to invoke the main() method. So, it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] x or String x[]** is used for command line argument.
- **System.out.println()** is used to print statement. Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class.

Difference between JDK, JRE, and JVM:**JVM:**

JVM (Java Virtual Machine) is an abstract machine. It is called a virtual machine because it doesn't physically exist. It is a specification that provides a runtime environment in which Java bytecode can be executed. It can also run those programs which are written in other languages and compiled to Java bytecode.

JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent. There are three notions of the JVM: specification, implementation, and instance.

The JVM performs the following main tasks:

Loads code

Verifies code

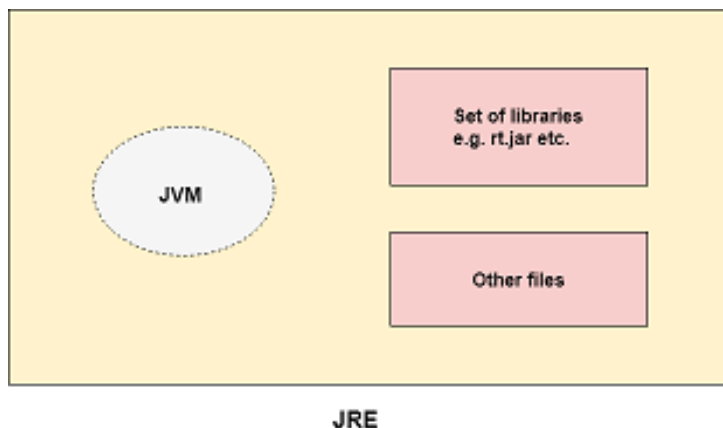
Executes code

Provides runtime environment

JRE

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.

The implementation of JVM is also actively released by other companies besides Sun Micro Systems.



DK

JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools.

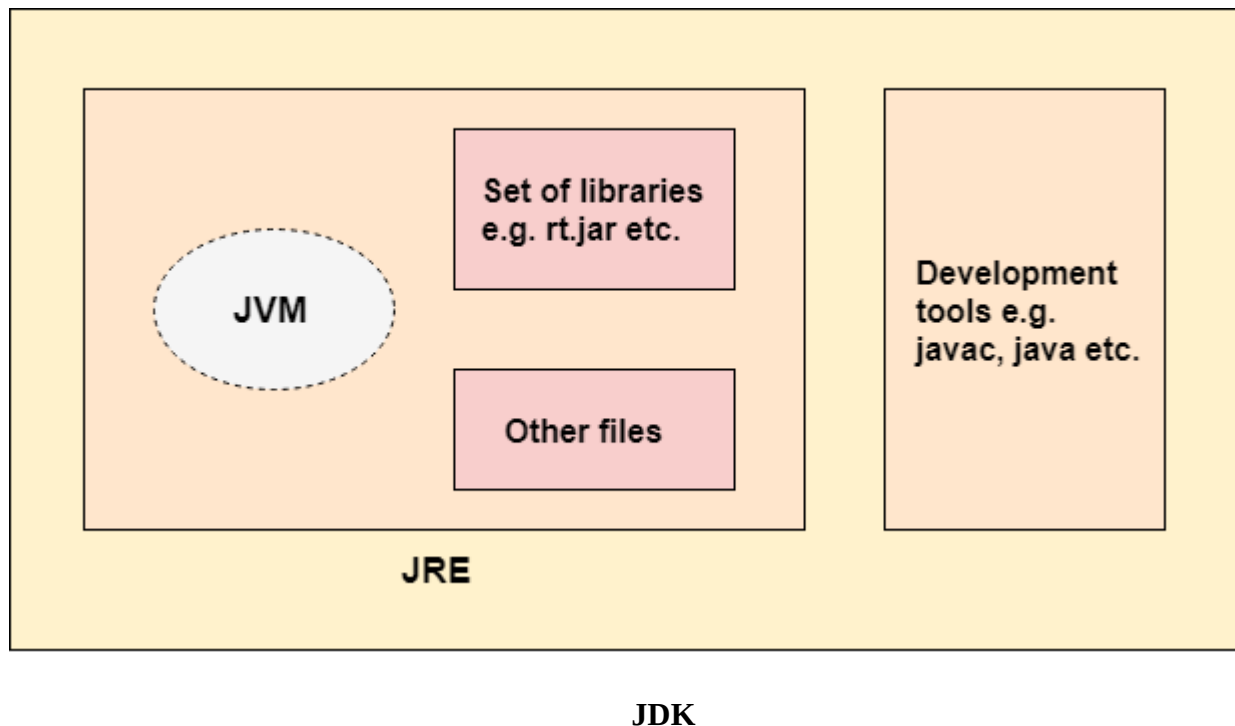
JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

Standard Edition Java Platform

Enterprise Edition Java Platform

Micro Edition Java Platform

The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.



Java OOPs Concepts

In this page, we will learn about the basics of OOPs. Object-Oriented Programming is a paradigm that provides many concepts, such as inheritance, data binding, polymorphism, etc.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language. Smalltalk is considered the first truly object-oriented programming language.

The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.

The main aim of object-oriented programming is to implement real-world entities, for example, object, classes, abstraction, inheritance, polymorphism, etc.

OOPs (Object-Oriented Programming System)

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Apart from these concepts, there are some other terms which are used in Object-Oriented design:

- Coupling
- Cohesion
- Association
- Aggregation
- Composition

Types of Variables:

- Based on the type of value represented by the variable all variables are divided into 2 types. They are:
 - 1) Primitive variables
 - 2) Reference variables

Primitive variables: Primitive variables can be used to represent primitive values.

Example: `int x=10;`

Reference variables: Reference variables can be used to refer objects.

Example: `Student s=new Student();`

- Based on the purpose and position of declaration all variables are divided into the following 3 types.
 - 1) Instance variables
 - 2) Static variables
 - 3) Local variables

Instance variables:

- If the value of a variable is varied from object to object such type of variables are called instance variables.
- For every object a separate copy of instance variables will be created.
- Instance variables will be created at the time of object creation and destroyed at the time of object destruction hence the scope of instance variables is exactly same as scope of objects.
- Instance variables will be stored on the heap as the part of object.
- Instance variables should be declared with in the class directly but outside of any method or block or constructor.
- Instance variables can be accessed directly from Instance area. But cannot be accessed directly from static area.
- But by using object reference we can access instance variables from static area.

Static variables:

- If the value of a variable is not varied from object to object such type of variables is not recommended to declare as instance variables. We have to declare such type of variables at class level by using static modifier.
- In the case of instance variables for every object a separate copy will be created but in the case of static variables for entire class only one copy will be created and shared by every object of that class.
- Static variables will be created at the time of class loading and destroyed at the time of class unloading hence the scope of the static variable is exactly same as the scope of the **.class** file.
- Static variables will be stored in method area. Static variables should be declared with in the class directly but outside of any method or block or constructor.
- Static variables can be accessed from both instance and static areas directly.
- We can access static variables either by class name or by object reference but usage of class name is recommended.
- But within the same class it is not required to use class name we can access directly.
 - 1) Start JVM.
 - 2) Create and start Main Thread by JVM.
 - 3) Locate(find) Test.class by main Thread.
 - 4) Load Test.class by main Thread.
 - 5) Execution of main() method.
 - 6) Unload Test.class
 - 7) Terminate main Thread.
 - 8) Shutdown JVM.

- For the static variables it is not required to perform initialization explicitly, JVM will always provide default values.
- Static variables also known as class level variables or fields.

Local variables:

- Some time to meet temporary requirements of the programmer we can declare variables inside a method or block or constructors such type of variables are called local variables or automatic variables or temporary variables or stack variables.
- The local variables will be created as part of the block execution in which it is declared and destroyed once that block execution completes. Hence the scope of the local variables is exactly same as scope of the block in which we declared.
- For the local variables JVM won't provide any default values compulsory we should perform initialization explicitly before using that variable.

Week-2

Object: Program illustrating Classes, Objects and Method Overloading.

Object: Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

An object has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Class: Collection of objects is called class. It is a logical entity. A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

Inheritance: When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism: If one task is performed in different ways, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

In Java, we use method overloading and method overriding to achieve polymorphism.

Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

Abstraction: Hiding internal details and showing functionality is known as abstraction. For example, phone call, we don't know the internal processing.

in Java, we use abstract class and interface to achieve abstraction.

Encapsulation: Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines. A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Class in Java:

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- Fields
- Methods
- Constructors
- Blocks
- Nested class and interface

Object and Class Example: main within the class

```
class Student
{
    int id;                //field or data member or instance variable
    String name;
    public static void main(String args[])
    {
        Student s1=new Student();    //creating an object of Student
        System.out.println(s1.id);    //accessing member through reference variable
        System.out.println(s1.name);
    }
}
```

Output:

```
0
null
```

Object and Class Example: Initialization through method:

```
class Student
{
    int rollno;
    String name;
    void insertRecord(int r, String n)
    {
        rollno=r;
        name=n;
    }
    void displayInformation()
    {
        System.out.println(rollno+" "+name);
    }
}
class TestStudent4
{
    public static void main(String args[])
    {
        Student s1=new Student();
        Student s2=new Student();
        s1.insertRecord(111,"Karan");
        s2.insertRecord(222,"Aryan");
        s1.displayInformation();
        s2.displayInformation();
    }
}
```

Output:

```
111 Karan
222 Aryan
```

Example: Java Program to demonstrate the working of a banking-system where we deposit and withdraw amount from our account. Creating an Account class which has deposit () and withdraw () methods

```
class Account
{
    int acc_no;
    String name;
    float amount;
    void insert(int a,String n,float amt)
    {
        acc_no=a;
        name=n;
        amount=amt;
    }
    //deposit method
    void deposit(float amt)
    {
        amount=amount+amt;
        System.out.println(amt+" deposited");
    }
    //withdraw method
    void withdraw(float amt)
    {
        if(amount<amt)
        {
            System.out.println("Insufficient Balance");
        }
        else
        {
```

```

        amount=amount-amt;

        System.out.println(amt+" withdrawn");
    }
}

//method to check the balance of the account
void checkBalance()
{
    System.out.println("Balance is: "+amount);}

//method to display the values of an object
void display()
{
    System.out.println(acc_no+" "+name+" "+amount);
}
}

//Creating a test class to deposit and withdraw amount
class TestAccount
{
    public static void main(String[] args)
    {
        Account a1=new Account();
        a1.insert(832345,"Ankit",1000);
        a1.display();
        a1.checkBalance();
        a1.deposit(40000);
        a1.checkBalance();
        a1.withdraw(15000);
        a1.checkBalance();
    }
}

```

Constructors in Java :

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called. It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

Types of constructors in Java: no-arg constructor, and parameterized constructor.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Default constructor:

- For every class in java including abstract classes also constructor concept is applicable.
- If we are not writing at least one constructor then compiler will generate default constructor.
- If we are writing at least one constructor then compiler won't generate any default constructor. Hence every class contains either compiler generated constructor (or) programmer written constructor but not both simultaneously.

Example of default Constructor:

```
class Student3
{
    int id;
    String name;
    void display()
    {
        System.out.println(id+" "+name);
    }

    public static void main(String args[])
    {
        Student3 s1=new Student3();
        Student3 s2=new Student3();
        s1.display();
        s2.display();
    }
}
```

Output

0 null

0 null

Java Parameterized Constructor:

A constructor which has a specific number of parameters is called a parameterized constructor.

Use the parameterized constructor:

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor:

```
class Student4
{
    int id;
    String name;
    //creating a parameterized constructor
    Student4(int i,String n)
    {
        id = i;
        name = n;
    }
    //method to display the values
    void display()
    {
        System.out.println(id+" "+name);
    }

    public static void main(String args[])
    {
        //creating objects and passing values
        Student4 s1 = new Student4(111,"Karan");
```



```

        Student4 s2 = new Student4(222,"Aryan");
        //calling method to display the values of object
        s1.display();
        s2.display();
    }
}

```

Rules for creating Java Constructor:

- Constructor name must be the same as its class name.
- A Constructor must have no explicit return type.
- A Java constructor cannot be abstract, static, final, and synchronized.

Example: -

```

class Student
{
    String name; int rollno;
    Student(String name,int rollno)\\Constructor
    {
        this.name=name;

        this.rollno=rollno;
    }
    public static void main(String[] args)
    {
        Student s1=new Student("vijayabhaskar",101);
        Student s2=new Student("bhaskar",102);
    }
}

```

Output:

```

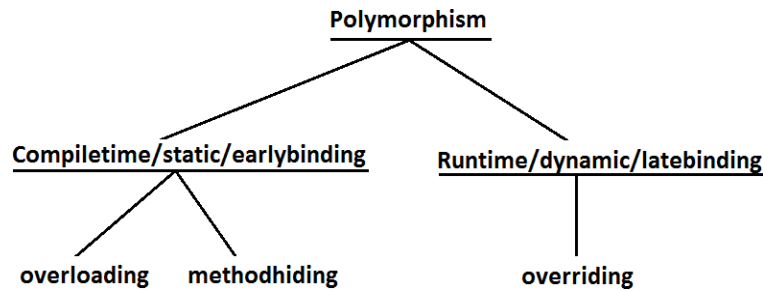
111 Karan
222 Aryan

```

Polymorphism in Java:

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.



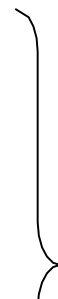
- 1) Inheritance talks about reusability.
- 2) Polymorphism talks about flexibility.
- 3) Encapsulation talks about security.

Overloading:

- Two methods are said to be overload if and only if both having the same name but different argument types.
- In 'C' language we can't take 2 methods with the same name and different types. If there is a change in argument type compulsory, we should go for new method name.
- Lack of overloading in "C" increases complexity of the programming.
- But in java we can take multiple methods with the same name and different argument types.

Example:

```
class Test
{
    public void methodOne()
    {
```



```

        System.out.println("no-arg method");
    }
    public void methodOne(int i)
    {
        System.out.println("int-arg method");
    }
    public void methodOne(double d)
    {
        System.out.println("double-arg method");
    }
    public static void main(String[] args)
    {
        Test t=new Test();
        t.methodOne();//no-arg
        method
        t.methodOne(10);//int-
        arg method
        t.methodOne(10.5);//double-arg method
    }
}

```

overloaded methods

Note: -In overloading compiler is responsible to perform method resolution(decision) based on the reference type. Hence overloading is also considered as compile time polymorphism(or) static () early binding.

Week 3

Inheritance in Java :

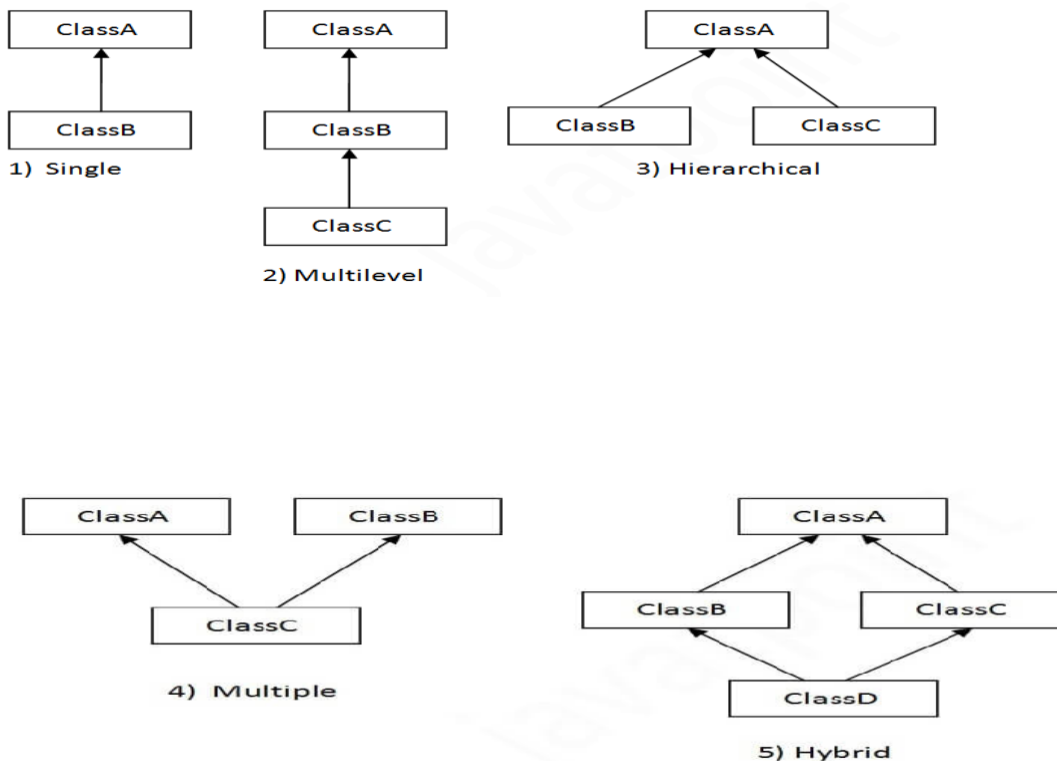
Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system). The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also. Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Why use inheritance in java:

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Types of inheritance in java:

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.



Example of single Inheritance:

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
class TestInheritance
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.bark();
        d.eat();
    }
}
```

Output:

```
barking...
eating...
```

Multilevel Inheritance Example:

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
class BabyDog extends Dog
{
    void weep()
    {
        System.out.println("weeping...");
    }
}
class TestInheritance2
{
    public static void main(String args[])
    {
        BabyDog d=new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}
```

Output:

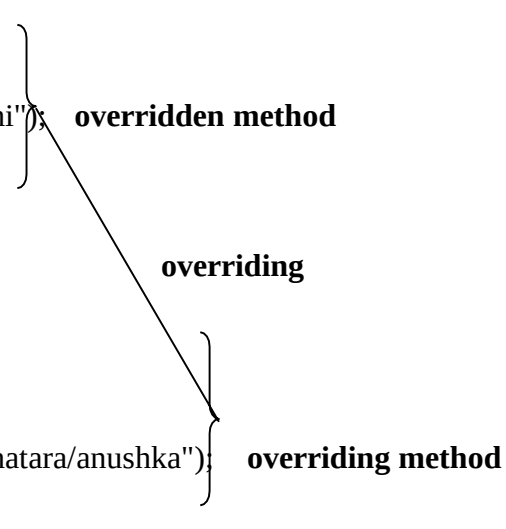
```
weeping...
barking...
eating...
```

Overriding:

- Whatever the Parent has by default available to the Child through inheritance, if the Child is not satisfied with Parent class method implementation, then Child is allow to redefine that Parent class method in Child class in its own way this process is called overriding.
- The Parent class method which is overridden is called overridden method.
- The Child class method which is overriding is called overriding method.

Example 1:

```
class Parent
{
    public void property()
    {
        System.out.println("cash+land+gold");
    }
    public void marry()
    {
        System.out.println("subbalakshmi");
    }
}
class Child extends Parent
{
    public void marry()
    {
        System.out.println("Trisha/nayanatara/anushka");
    }
}
class Test
{
    public static void main(String[] args)
    {
        Parent p=new Parent();
        p.marry();//subbalakshmi(paret method)
        Child c=new Child();
        c.marry();//Trisha/nayanatara/anushka(child method)
        Parent p1=new Child();
        p1.marry();//Trisha/nayanatara/anushka(child method)
    }
}
```



- In overriding method resolution is always takes care by JVM based on runtime object hence overriding is also considered as runtime polymorphism or dynamic polymorphism or late binding.
- The process of overriding method resolution is also known as dynamic method dispatch.

Note: In overriding runtime object will play the role and reference type is dummy.

Rules for overriding:

- In overriding method names and arguments must be same. That is method signature must be same.
- Until 1.4 version the return types must be same but from 1.5 version onwards co-variant return types are allowed.
- According to this Child class method return type need not be same as Parent class method return type its Child types also allowed.

Abstraction in Java:

Abstraction is a process of hiding the implementation details and showing only functionality to the user. Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class in Java:

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember:

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructor and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example:

```
abstract class Shape
{
    abstract void draw();
}
//In real scenario, implementation is provided by others i.e. unknown by end user

class Rectangle extends Shape
{
    void draw()
    {
        System.out.println("drawing rectangle");
    }
}
class Circle1 extends Shape
{
    void draw()
    {
```

```

        System.out.println("drawing circle");
    }
}
//In real scenario, method is called by programmer or user
class TestAbstraction1
{
    public static void main(String args[])
    {
        Shape s=new Circle1();
        s.draw();
    }
}

```

Interface in Java

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve abstraction*. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Java Interface also **represents the IS-A relationship**.

It cannot be instantiated just like the abstract class.

//Interface declaration: by first user

interface Drawable

```

{
    void draw();
}

```

//Implementation: by second user

class Rectangle **implements** Drawable

```

{
    public void draw()
    {
        System.out.println("drawing rectangle");
    }
}

```

class Circle **implements** Drawable

```

{
    public void draw()
    {
        System.out.println("drawing circle");
    }
}

```

//Using interface: by third user

```
class TestInterface1
{
    public static void main(String args[])
    {
        Drawable d=new Circle();//In real scenario, object is provided by method e.g. getDrawable()
        d.draw();
    }
}
```

Output:

drawing circle

drawing circle

Difference between abstract class and interface:

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated.

Sr.No	Abstract class	Interface
1	Abstract class can have abstract and non-abstract methods .	Interface can have only abstract methods . Since Java 8 , it can have default and static methods also .
2	Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3	Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4	Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5	The abstract keyword is used to declare abstract class .	The interface keyword is used to declare interface .
6	An abstract class can extend another Java class and implement multiple Java interfaces .	An interface can extend another Java interface only.
7	An abstract class can be extended using keyword "extends" .	An interface can be implemented using keyword "implements" .
8	A Java abstract class can have class members like private, protected, etc .	Members of a Java interface are public by default.

Example of Interface:

interface A

```
{  
    void a();//bydefault, public and abstract  
    void b();  
    void c();  
    void d();  
}
```

abstract class B **implements** A

```
{  
    public void c()  
    {  
        System.out.println("I am C");  
    }  
}
```

class M **extends** B

```
{  
    public void a()  
    {  
        System.out.println("I am a");  
    }  
    public void b()  
    {  
        System.out.println("I am b");  
    }  
    public void d()  
    {  
        System.out.println("I am d");  
    }  
}
```

//Creating a test class that calls the methods of A interface

class Test5

```
{  
    public static void main(String args[])  
    {  
        A a=new M();  
        a.a();  
        a.b();  
        a.c();  
        a.d();  
    }  
}
```

Output:

I am a

I am b

I am c

I am d

Week-4

Object: 1- Program illustrating use of Final and Super keyword.

2- Program that illustrates the following

- a) Creation of simple package.
- b) Accessing a package.

Final Keyword in Java:

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many contexts. Final can be:

- 1. variable
- 2. method
- 3. class

Java final variable:

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example of final variable

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

Example:-

```
class Bike9
{
    final int speedlimit=90;//final variable
    void run()
    {
        speedlimit=400;
    }
    public static void main(String args[])
    {
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

Output:

Compile Time Error

2) Java final method:

If you make any method as final, you cannot override it.

Example of final method:

```
class Bike
{
    final void run()
    {
        System.out.println("running");
    }
}

class Honda extends Bike
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
    public static void main(String args[])
    {
        Honda honda= new Honda();
        honda.run();
    }
}
```

Output:

Compile Time Error

Java final class:

If you make any class as final, you cannot extend it.

Example of final class:

```
final class Bike
{
}

class Honda1 extends Bike{
    void run()
```



```

    {
    System.out.println("running safely with 100kmph");
    }
    public static void main(String args[])
    {
        Honda1 honda= new Honda1();
        honda.run();
    }
}

```

Output:

Compile Time Error

Super Keyword in Java:

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

Example: super is used to refer immediate parent class instance variable

```

class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void printColor()
    {
        System.out.println(color);//prints color of Dog class
        System.out.println(super.color);//prints color of Animal class
    }
}

```

```

    }
}
class TestSuper1
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.printColor();
    }
}

```

Output:

```

black
white

```

Example 2: super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```

class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void eat()
    {
        System.out.println("eating bread...");
    }
    void bark()
    {
        System.out.println("barking...");
    }
    void work()
    {
        super.eat();
    }
}

```

```

        bark();
    }
}
class TestSuper2
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.work();
    }
}

```

Output:

```

eating...
barking...

```

Example 3: super is used to invoke parent class constructor

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

```

class Animal
{
    Animal()
    {
        System.out.println("animal is created");
    }
}
class Dog extends Animal
{
    Dog()
    {
        super();
        System.out.println("dog is created");
    }
}
class TestSuper3
{
    public static void main(String args[])
    {
        Dog d=new Dog();
    }
}

```

➤ Output:

```
animal is created  
dog is created
```

Java Package:

A **java package** is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc. Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package:

- Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- Java package provides access protection.
- Java package removes naming collision.

Example of java package

The **package keyword** is used to create a package in java.

```
package mypack;  
public class Simple  
{  
    public static void main(String args[])  
    {  
        System.out.println("Welcome to package");  
    }  
}
```

Week-5

Object: Program for creating multiple threads

- Using Thread class.
- Using Runnable Interface

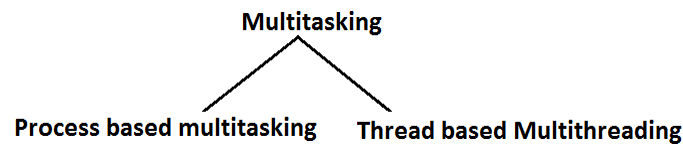
Multithreading:

Multithreading in java is a process of executing multiple threads simultaneously. A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

Multitasking: Executing several tasks simultaneously is the concept of multitasking. There are two types of multitasking's.

- 1) **Process based multitasking.**
- 2) **Thread based multitasking.**

Diagram:



Process based multitasking: Executing several tasks simultaneously where each task is a separate independent process such type of multitasking is called process based multitasking.

Example:

- While typing a java program in the editor we can able to listen mp3 audio songs at the same time we can download a file from the net all these tasks are independent of each other and executing simultaneously and hence it is Process based multitasking.
- This type of multitasking is best suitable at “**os level**”.

Thread based multitasking: Executing several tasks simultaneously where each task is a separate independent part of the same program, is called Thread based multitasking. And each independent part is called a “Thread”.

- This type of multitasking is best suitable for “programatic level”.
- When compared with “C++”, developing multithreading examples is very easy in java because java provides in built support for multithreading through a rich API (Thread, Runnable, ThreadGroup, ThreadLocal....etc).
- In multithreading on 10% of the work the programmer is required to do and 90% of the work will be down by java API.
- The main important application areas of multithreading are:
 - 1) To implement multimedia graphics.
 - 2) To develop animations.
 - 3) To develop video games etc.

- Whether it is process based or Thread based the main objective of multitasking is to improve performance of the system by reducing response time.

The Way to start instantiate and define a new Thread:

We can define a Thread in the following 2 ways.

1. By extending Thread class.
2. By implementing Runnable interface.

Defining a Thread by extending “Thread class”:

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();//Instantiation of
        a Thread t.start();//starting of a Thread
        ──────────>
        for(int i=0;i<5;i++)
        {
            System.out.println("main thread");
        }
    }
}
```

Case 1: Thread Scheduler:

- If multiple Threads are waiting to execute then which Thread will execute 1st is decided by “Thread Scheduler” which is part of JVM.
- Which algorithm or behavior followed by Thread Scheduler we can’t expect exactly it is the JVM vendor dependent hence in multithreading examples we can’t expect exact execution order and exact output.

Case 2: Difference between t.start() and t.run() methods.

In the case of t.start() a new Thread will be created which is responsible for the execution of run() method. But in the case of t.run() no new Thread will be created and run() method will be executed just like a normal method by the main Thread.

Case 3: importance of Thread class start() method.

For every Thread the required mandatory activities like registering the Thread with Thread Scheduler will take care by Thread class start() method and programmer is responsible just to define the job of the Thread inside run() method. That is start() method acts as best

assistant to the programmer.

Example:

```
start()
{
    1.    Register Thread with Thread Scheduler
    2.    All other mandatory low level activities.
    3.    Invoke or calling run() method.
}
```

- We can conclude that without executing Thread class start() method there is no chance of starting a new Thread in java.

Case 4: If we are not overriding run() method:

- If we are not overriding run() method then Thread class run() method will be executed which has empty implementation and hence we won't get any output.

Example:

```
class MyThread extends Thread
{
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new MyThread();
        t.start();
    }
}
```

Note: It is highly recommended to override run() method. Otherwise don't go for multithreading concept.

Case 5: Overriding of run() method.

We can overload run() method but Thread class start() method always invokes no argument run() method the other overload run() methods we have to call explicitly then only it will be executed just like normal method.

Example:

```
class MyThread extends Thread
{
    public void run()
    {
```

```

        System.out.println("no arg method");
    }
    public void run(int i)
    {
        System.out.println("int arg method");
    }
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new
        MyThread(); t.start();
    }
}

```

Output:

No arg method

Case 6: overriding of start() method:

If we override start() method then our start() method will be executed just like a normal method call and no new Thread will be started.

Example:

```

class MyThread extends Thread
{
    public void start()
    {
        System.out.println("start method");
    }
    public void run()
    {
        System.out.println("run method");
    }
}
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t=new
        MyThread();
        t.start();
    }
}

```



```

        System.out.println("main method");
    }
}

```

Output:

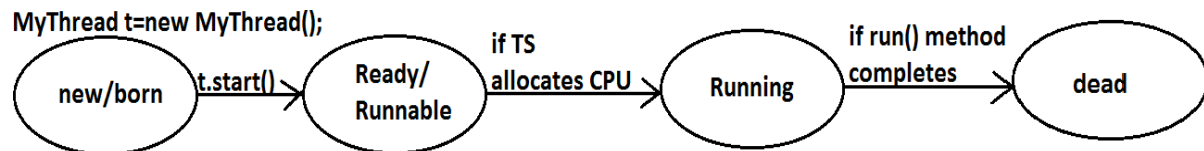
start method

main method

- Entire output produced by only main Thread.

Life cycle of the Thread:

Diagram:

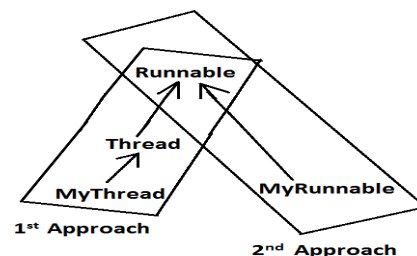


- Once we created a Thread object then the Thread is said to be in new state or born state.
- Once we call start() method then the Thread will be entered into Ready or Runnable state.
- If Thread Scheduler allocates CPU then the Thread will be entered into running state.
- Once run() method completes then the Thread will entered into dead state.

Defining a Thread by implementing Runnable interface:

- We can define a Thread even by implementing Runnable interface also. Runnable interface present in java.lang.pkg and contains only one method run().

Diagram:



Example:

defining a Thread

```
class MyRunnable implements Runnable
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("child Thread");
        }
    }
}
```

job of a Thread

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyRunnable r=new MyRunnable();
        Thread t=new Thread(r);//here r is a Target
        Runnable t.start();
        → for(int i=0;i<10;i++)
        {
            System.out.println("main thread");
        }
    }
}
```

Week-6

Object: Program that illustrates the following

- Handling predefined exceptions.
- Handling user defined exceptions.

Exception Handling in Java:

The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.

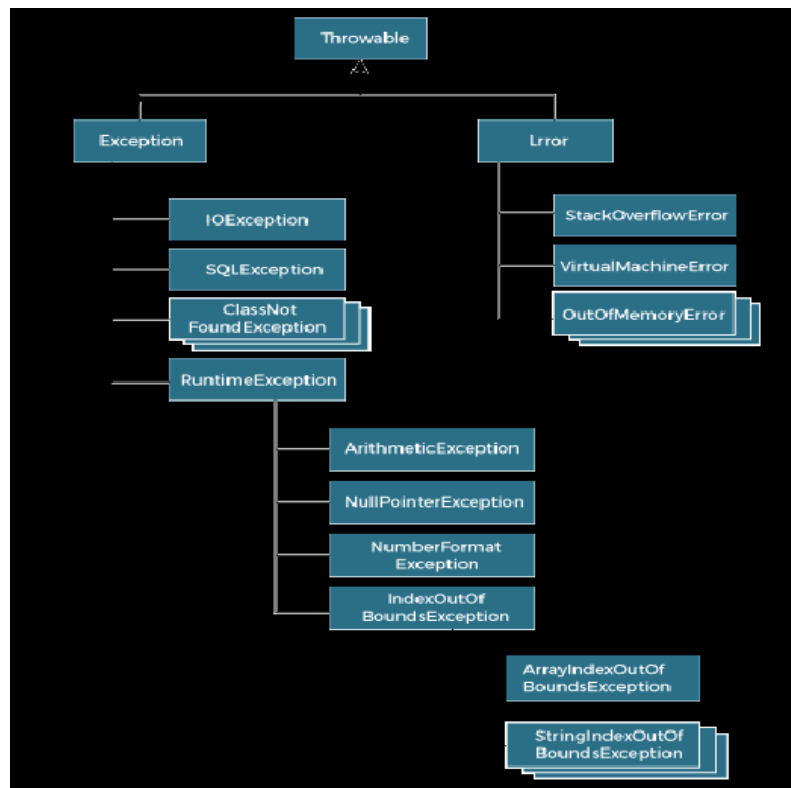
Exception in Java:

Dictionary Meaning: Exception is an abnormal condition.

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Hierarchy of Java Exception classes:

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions:

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Default exception handling in java:

- 1) If an exception raised inside any method then the method is responsible to create Exception object with the following information.
 - 1) Name of the exception.
 - 2) Description of the exception.
 - 3) Location of the exception.
- 2) After creating that Exception object the method handovers that object to the JVM.
- 3) JVM checks whether the method contains any exception handling code or not. If method won't contain any handling code then JVM terminates that method abnormally.
- 4) JVM identifies the caller method and checks whether the caller method contain any handling code or not. If the caller method also does not contain handling code then JVM terminates that caller also abnormally.
- 5) This process will be continued until main() method and if the main() method also doesn't contain any exception handling code then JVM terminates main() method . Then JVM handovers the responsibility of exception handling to the default exception handler.
- 6) Default exception handler just print exception information to the console in the following formats and terminates the program abnormally.
 - Name of exception
 - Description
 - Location of exception (stack trace)

Java Exception Keywords:

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Exception Handling Example:

```
public class JavaExceptionExample
{
    public static void main(String args[])
    {
        Try
        {
            int data=100/0;           //code that may raise exception
        }
        catch(ArithmeticException e)
        {
            System.out.println(e);
        }
        //rest code of the program
        System.out.println("rest of the code...");
    }
}
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

Java try-catch block:

Java try block:

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Java catch block:

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Java Multi-catch block:

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Example of Multiple catch Block:

```
public class MultipleCatchBlock1
{
    public static void main(String[] args)
    {
        Try
        {
            int a[]=new int[5];
            a[5]=30/0;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception occurs");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("ArrayIndexOutOfBounds Exception occurs");
        }
        catch(Exception e)
        {
            System.out.println("Parent Exception occurs");
        }
        System.out.println("rest of the code");
    }
}
```

Output:

```
Arithmetic Exception occurs
rest of the code
```

Java finally block

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

Use Java finally block?

- finally block in Java can be used to put "**cleanup**" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

Example of Finally:

```
class TestFinallyBlock
{
    public static void main(String args[])
    {
        Try
        {
            //below code do not throw any exception
            int data=25/5;
            System.out.println(data);
        }
        //catch won't be executed
        catch(NullPointerException e)
        {
            System.out.println(e);
        }
        //executed regardless of exception occurred or not
        Finally
        {
            System.out.println("finally block is always executed");
        }

        System.out.println("rest of phe code...");
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock.java
C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock
5
finally block is always executed
rest of the code...
```


Java throw Exception:

In Java, exceptions allows us to write good quality codes where the errors are checked at the compile time instead of runtime and we can create custom exceptions making the code recovery and debugging easier.

Java throw keyword:

The Java throw keyword is used to throw an exception explicitly.

We specify the **exception** object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.

We can throw either checked or unchecked exceptions in Java by throw keyword. It is mainly used to throw a custom exception. We will discuss custom exceptions later in this section.

We can also define our own set of conditions and throw an exception explicitly using throw keyword. For example, we can throw ArithmeticException if we divide a number by another number. Here, we just need to set the condition and throw exception using throw keyword.

```
public class TestThrow1
{
    public static void validate(int age)    //function to check if person is eligible to vote or not
    {
        if(age<18)
        {
            //throw Arithmetic exception if not eligible to vote
            throw new ArithmeticException("Person is not eligible to vote");
        }
        else
        {
            System.out.println("Person is eligible to vote!!");
        }
    }
    public static void main(String args[])
    {
        validate(13);    //calling the function
        System.out.println("rest of the code...");
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow1.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow1
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to
vote
    at TestThrow1.validate(TestThrow1.java:8)
    at TestThrow1.main(TestThrow1.java:18)
```

Java throws keyword

The **Java throws keyword** is used to declare an exception. It gives an information to the programmer that there may occur an exception. So, it is better for the programmer to provide the exception handling code so that the normal flow of the program can be maintained.

Exception Handling is mainly used to handle the checked exceptions. If there occurs any unchecked exception such as `NullPointerException`, it is programmers' fault that he is not checking the code before it being used.

Which exception should be declared?

Ans: Checked exception only, because:

- **unchecked exception:** under our control so we can correct our code.
- **error:** beyond our control. For example, we are unable to do anything if there occurs `VirtualMachineError` or `StackOverflowError`.

Advantage of Java throws keyword:

Now Checked Exception can be propagated (forwarded in call stack).

It provides information to the caller of the method about the exception.

Difference between final, finally, and finalize:

Final:

- Final is the modifier applicable for class, methods and variables.
- If a class declared as the final then child class creation is not possible.
- If a method declared as the final then overriding of that method is not possible.
- If a variable declared as the final then reassignment is not possible.

➤ Finally:

- It is the block always associated with try catch to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether handled or not handled.

Finalize:

- It is a method which should be called by garbage collector always just before destroying an object to perform cleanup activities.

Note: To maintain clean up code faunally block is recommended over finalize() method because we can't expert exact behavior of GC.

Example of throws:

```
import java.io.*;
class M
{
    void method()throws IOException
    {
        throw new IOException("device error");
    }
}
class Testthrows4
{
    public static void main(String args[])throws IOException{//declare exception
        M m=new M();
        m.method();

        System.out.println("normal flow...");
    }
}
```

Output:

```
Exception in thread "main" java.io.IOException: device error
    at M.method(Testthrows4.java:4)
    at Testthrows4.main(Testthrows4.java:10)
```

Week-7

Object: Program to illustrate the concept of Applets and Incorporating Graphics.

Java Applet: Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet:

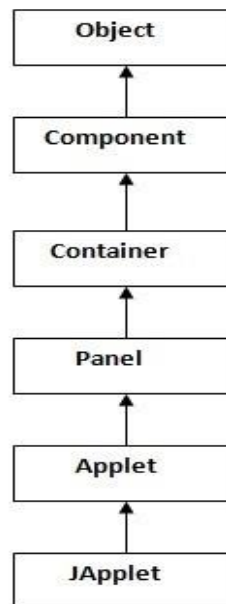
There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Drawback of Applet:

- Plugin is required at client browser to execute applet.

Hierarchy of Applet:



Lifecycle of Java Applet:

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

Lifecycle methods for Applet:

The java.applet.Applet class provides 4 life cycle methods and java.awt.Component class provides 1 life cycle method for an applet.

java.applet.Applet class

For creating any applet java.applet.Applet class must be inherited. It provides 4 life cycle methods of applet.

1. **public void init():** is used to initialize the Applet. It is invoked only once.
2. **public void start():** is invoked after the init() method or browser is maximized. It is used to start the Applet.
3. **public void stop():** is used to stop the Applet. It is invoked when Applet is stopped or browser is minimized.
4. **public void destroy():** is used to destroy the Applet. It is invoked only once.

java.awt.Component class

The Component class provides 1 life cycle method of applet.

1. **public void paint(Graphics g):** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Running an Applet:

There are two ways to run an applet

1. By html file.
2. By appletViewer tool (for testing purpose).

Example of Applet by html file:

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("welcome",150,150);
    }
}
```

myapplet.html

```
<html>
<body>
<applet code="First.class" width="300" height="300">
</applet>
</body>
</html>
```

Example of Applet by appletviewer tool

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{

    public void paint(Graphics g){
        g.drawString("welcome to applet",150,150);
    }

}
/*
<applet code="First.class" width="300" height="300">
</applet>
*/
```

To execute the applet by appletviewer tool, write in command prompt:

```
c:\>javac First.java
c:\>appletviewer First.java
```

Displaying Graphics in Applet :

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```
import java.applet.Applet;
import java.awt.*;
public class GraphicsDemo extends Applet
{
    public void paint(Graphics g)
    {
        g.setColor(Color.red);
        g.drawString("Welcome",50, 50);
        g.drawLine(20,30,20,300);
        g.drawRect(70,100,30,30);
        g.fillRect(170,100,30,30);
        g.drawOval(70,200,30,30);
        g.setColor(Color.pink);
        g.fillOval(170,200,30,30);
    }
}
```

```

        g.drawArc(90,150,30,30,30,270);
        g.fillArc(270,150,30,30,0,180);
    }

```

myapplet.html

```

<html>
<body>
<applet code="GraphicsDemo.class" width="300" height="300">
</applet>
</body>
</html>

```

Java AWT Panel:

The Panel is a simplest container class. It provides space in which an application can attach any other component. It inherits the Container class.

It doesn't have title bar.

Java AWT Panel Example

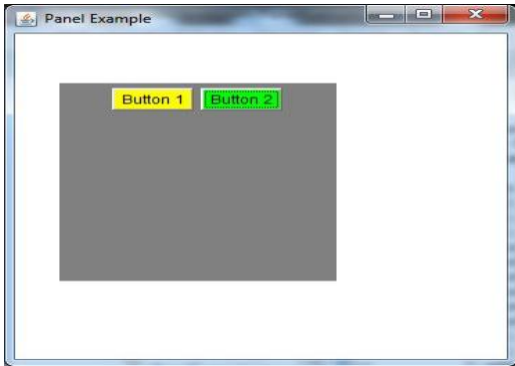
```

import java.awt.*;
public class PanelExample {
    PanelExample()
    {
        Frame f= new Frame("Panel Example");
        Panel panel=new Panel();
        panel.setBounds(40,80,200,200);
        panel.setBackground(Color.gray);
        Button b1=new Button("Button 1");
        b1.setBounds(50,100,80,30);
        b1.setBackground(Color.yellow);
        Button b2=new Button("Button 2");
        b2.setBounds(100,100,80,30);
        b2.setBackground(Color.green);
        panel.add(b1); panel.add(b2);
        f.add(panel);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new PanelExample();
    }
}

```


}

Output:



Java LayoutManagers

The LayoutManagers are used to arrange components in a particular manner. The **Java LayoutManagers** facilitates us to control the positioning and size of the components in GUI forms. LayoutManager is an interface that is implemented by all the classes of layout managers. There are the following classes that represent the layout managers:

1. java.awt.BorderLayout
2. java.awt.FlowLayout
3. java.awt.GridLayout
4. java.awt.CardLayout
5. java.awt.GridBagLayout
6. javax.swing.BoxLayout
7. javax.swing.GroupLayout
8. javax.swing.ScrollPaneLayout
9. javax.swing.SpringLayout etc.

Java BorderLayout

The BorderLayout is used to arrange the components in five regions: north, south, east, west, and center. Each region (area) may contain one component only. It is the default layout of a frame or window. The BorderLayout provides five constants for each region:

1. **public static final int NORTH**
2. **public static final int SOUTH**
3. **public static final int EAST**
4. **public static final int WEST**
5. **public static final int CENTER**

Constructors of BorderLayout class:

- **BorderLayout():** creates a border layout but with no gaps between the components.
- **BorderLayout(int hgap, int vgap):** creates a border layout with the given horizontal and vertical gaps between the components.

Example:

// import statements

import java.awt.*;

import javax.swing.*;

public class BorderLayoutWithoutRegionExample

{

JFrame jframe;

// constructor

BorderLayoutWithoutRegionExample()

{

jframe = new JFrame();

JButton btn1 = new JButton("NORTH");

JButton btn2 = new JButton("SOUTH");

JButton btn3 = new JButton("EAST");

JButton btn4 = new JButton("WEST");

JButton btn5 = new JButton("CENTER");

// horizontal gap is 7, and the vertical gap is 7

// Since region is not specified, the gaps are of no use

jframe.setLayout(new BorderLayout(7, 7));

// each button covers the whole area

// however, the btn5 is the latest button

// that is added to the frame; therefore, btn5

// is shown

jframe.add(btn1);

jframe.add(btn2);

jframe.add(btn3);

jframe.add(btn4);

jframe.add(btn5);

jframe.setSize(300,300);

jframe.setVisible(true);

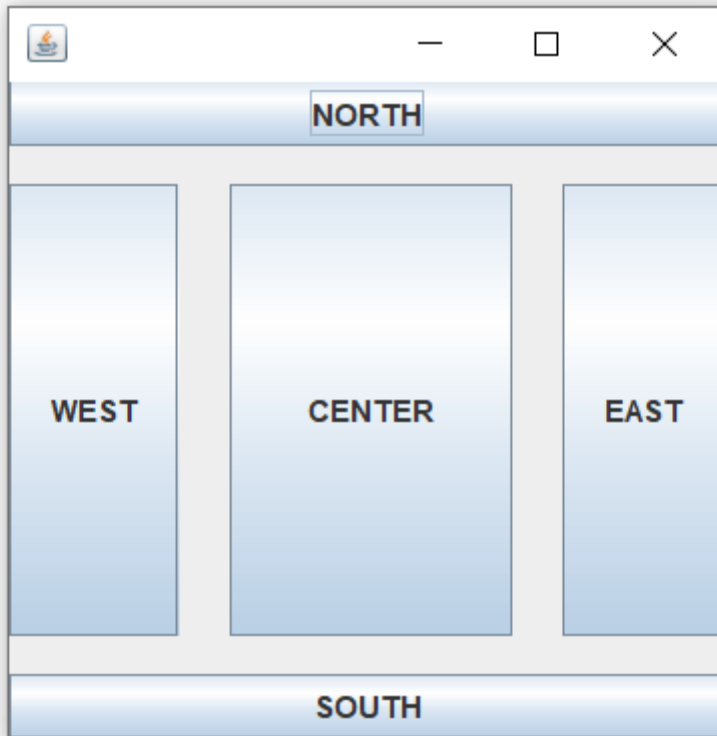
}

// main method

public static void main(String argsv[])

```
{  
    new BorderLayoutWithoutRegionExample();  
}
```

Output:



Week-8

Object: Working with Colors and Fonts.

Problem Description

How to fill colors in shapes using Applet?

Solution

Following example demonstrates how to create an applet which will have fill color in a rectangle using `setColor()`, `fillRect()` methods of `Graphics` class to fill color in a Rectangle.

```
import java.applet.*;

import java.awt.*;

public class fillColor extends Applet
{
    public void paint(Graphics g)
    {
        g.drawRect(300,150,200,100);
        g.setColor(Color.yellow);
        g.fillRect( 300,150, 200, 100 );
        g.setColor(Color.magenta);
        g.drawString("Rectangle",500,150);
    }
}
```

/*Java Program to Create and Fill Shapes using Applet*/

```
import java.applet.*;

import java.awt.*;

public class Shapes extends Applet
{
    //Function to initialize the applet
```

```

public void init()
{
    setBackground(Color.white);
}

//Function to draw and fill shapes
public void paint(Graphics g)
{
    //Draw a square
    g.setColor(Color.black);
    g.drawString("Square",75,200);
    int x[]={50,150,150,50};
    int y[]={50,50,150,150};
    g.drawPolygon(x,y,4);
    g.setColor(Color.yellow);
    g.fillPolygon(x,y,4);

    //Draw a pentagon
    g.setColor(Color.black);
    g.drawString("Pentagon",225,200);
    x=new int[]{200,250,300,300,250,200};
    y=new int[]{100,50,100,150,150,100};
    g.drawPolygon(x,y,6);
    g.setColor(Color.yellow);
    g.fillPolygon(x,y,6);

    //Draw a circle
    g.setColor(Color.black);
    g.drawString("Circle",400,200);

```

```

g.drawOval(350,50,125,125);
g.setColor(Color.yellow);
g.fillOval(350,50,125,125);
//Draw an oval
g.setColor(Color.black);
g.drawString("Oval",100,380);
g.drawOval(50,250,150,100);
g.setColor(Color.yellow);
g.fillOval(50,250,150,100);
//Draw a rectangle
g.setColor(Color.black);
g.drawString("Rectangle",300,380);
x=new int[]{250,450,450,250};
y=new int[]{250,250,350,350};
g.drawPolygon(x,y,4);
g.setColor(Color.yellow);
g.fillPolygon(x,y,4);
//Draw a triangle
g.setColor(Color.black);
g.drawString("Traingle",100,525);
x=new int[]{50,50,200};
y=new int[]{500,400,500};
g.drawPolygon(x,y,3);
g.setColor(Color.yellow);
g.fillPolygon(x,y,3);
}

```

```
}
```

```
/*
```

```
<applet code = Shapes.class width=600 height=600>
```

```
</applet>
```

```
*/
```